Princípios de Entrada e Saída em Nível de Hardware

Professor: Paulo Roberto de Oliveira

Disciplina: Sistemas Operacionais

Departamento de Informática
Universidade Estadual de Maringá

Dispositivos de I/O

- Abaixo podemos ver alguns dispositivos e suas velocidades típicas de transmissão:

Teclado	10 bytes/seg
Mouse	100 bytes/seg
Modem 56K	7 KB/seg
Canal de telefone	8 KB/seg
Impressora laser	100 KB/seg
Scanner	400 KB/seg
Ethernet	1.25 MB/seg
USB	1.5 MB/seg
Disco IDE	5 MB/seg
CD-ROM 40x	6 MB/seg

Ethernet 100 Mbps	12.5 MB/seg
Bus ISA	16.7 MB/seg
Disco EIDE (ATA-2)	16.7 MB/seg
FireWire	50 MB/seg
Disco SCSI Ultra 2	80 MB/seg
Gigabit Ethernet	125 MB/seg
Bus PCI	528 MB/seg
Bus AGP	2,1 GB/seg
Sun Gigaplane XB backplane	20 GB/seg

Dispositivos de I/O

Categorias de dispositivos:

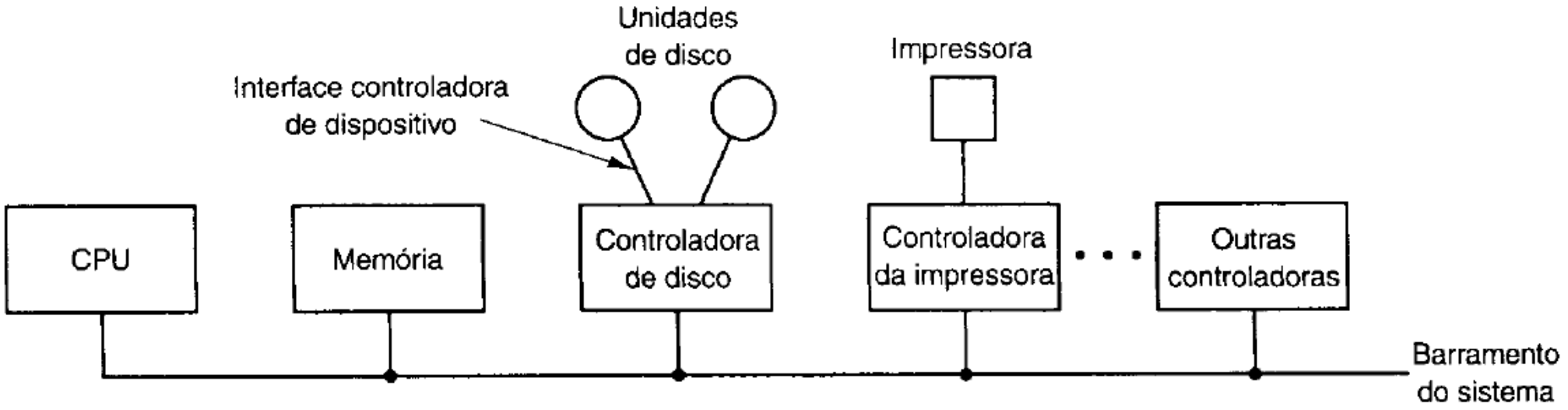
- **Bloco**: Armazena informações em blocos de tamanho fixo.
  - Ex.: Discos.
- **Caractere**: Entrega ou aceita *streams* de dados, sem considerar qualquer estrutura.
  - Ex.: Terminais, impressoras, redes, mouse, teclado.

Componentes de uma unidade de I/O:

- **Mecânico**: A unidade em si;
- **Eletrônico**: Controlador do dispositivo.

Dispositivos de I/O

- Os computadores modernos usam um sistema de **barramento único** para acessar todos os dispositivos.



- Mainframes usam múltiplos barramentos e computadores especializados em I/O (canais de I/O) para diminuir a carga da CPU principal.

Controladores de Dispositivos

<u>Exemplo</u>: Controladora de Disco.

- Quando se faz uma leitura, o que o disco devolve é um *stream* de bits!
- Composição do stream:
  - Um preâmbulo;
  - 16384 bits do setor lido;
  - *Checksum*.
- Preâmbulo: Escrito quando o disco é formatado;
  - Contém o cilindro, número e tamanho do setor.

Controladores de Dispositivos

- Função da controladora de disco:
  - Converter o *stream* de bits em um bloco de bytes;
    - O bloco é mantido em um buffer dentro da controladora.
  - Fazer correções de erros;
  - Declarar o bloco livre de erros.

Controladores de Dispositivos Registradores

- Cada controladora possui alguns registradores de controle:
  - Permitem a comunicação com a CPU;
  - O SO pode escrever neles para:
    - Enviar dados;
    - Receber dados;
    - Ligar ou desligar o dispositivo.
  - Lendo esses registradores, o SO pode:
    - Determinar o estado do dispositivo;
    - Verificar se ele pode aceitar um novo comando.

Controladores de Dispositivos Registradores

- O SO usa os registradores para passar comandos para a controladora;
- O SO dispara um comando e vai tratar de outras coisas;
- Quando a controladora termina um comando, ela interrompe a CPU para avisar que o que foi pedido está feito.
- Cada controladora possui um ou mais buffers, que o SO usa para escrever ou ler os dados das operações pedidas;

Controladores de Dispositivos
Registradores

- Há duas abordagens para a CPU se comunicar com os registradores das controladoras.
- Primeira: Cada registrador recebe um número de porta. Instruções especiais permitem ler ou gravar dados nesses registradores.

IN <REG> <PORTA>
Lê um dado da porta <PORTA> para o registrador <REG>

OUT <PORTA> <REG>
Escreve o conteúdo de <REG> na porta <PORTA>

Controladores de Dispositivos Mapeamento em Memória

- Segunda: Mapear todos os registradores de controle em memória. Cada registrador é associado a um endereço de memória que não é associado a uma posição de memória real.

Controladora de I/O	Endereços de I/O
Teclado	060-063
Disco	320-32F
Impressora	378-37F
Disquete	3F0-3F7

Controladores de Dispositivos Mapeamento em Memória

- Faixas de endereços são relacionadas com certos dispositivos;
- Em geral, as faixas são padronizadas;
- Se um fabricante altera a faixa de endereços de um dispositivo, todos os programas que trabalham com endereços de I/O (SO!) precisam ser modificados para rodar naquele computador.

Controladores de Dispositivos Mapeamento em Memória

- Um sistema misto, usado pelo Pentium, possui buffers de controladoras nos endereços de 640KB a 1MB, enquanto os registradores são mapeados em memória do endereço 0 a 64KB.

- Quais seriam as vantagens e desvantagens desses métodos?

Controladores de Dispositivos Mapeamento em Memória

- O I/O mapeado em memória permite melhor **proteção**, já que o SO pode evitar que um processo faça I/O não mapeando a região dos registradores no seu espaço de endereçamento.

- Se as controladoras estiverem em páginas **diferentes**, é possível dar um dispositivo a um usuário sem que ele possa acessar os demais.

Controladores de Dispositivos Mapeamento em Memória

- Com I/O mapeado em memória, instruções que testam memória também podem testar os registradores. Isso simplifica os programas.

- Se não, é preciso ler as portas para algum registrador antes de testá-lo.

- Uma desvantagem importante é que, hoje, a maioria dos computadores possui caches de memória. Colocar as páginas dos registradores em cache seria desastroso.

Controladores de Dispositivos Mapeamento em Memória

- Exemplo: Um programa testa um registrador – aquela página vai para o cache – um novo teste pega o valor lido anteriormente – quando o I/O for feito (e o registrador mudar), o programa não terá como descobrir que isso ocorreu.

- Uma solução para esse problema é desativar o cache para determinadas páginas. Isso obriga a se ter um hardware mais complexo, e o SO também.

DMA
(Direct Memory Access)

- Como a leitura de disco acontece **sem DMA**?
  1. A controladora lê um bloco (1 ou mais setores) do disco;
  2. Isso é feito bit a bit, até que o bloco inteiro esteja armazenado no buffer interno da controladora;
  3. Em seguida, ela computa o *checksum* para verificar se houve algum erro na leitura;
  4. A controladora causa uma interrupção;
  5. O SO realiza um loop lendo byte a byte (ou palavra a palavra) do buffer, através de um registrador da controladora, e transfere o bloco para a memória.

DMA
(Direct Memory Access)

- Problema: Perda de tempo de CPU!

- O DMA livra a CPU deste trabalho:
  - Ao fazer o pedido de leitura, a CPU fornece 3 parâmetros:
    - Endereço do bloco no disco (setor);
    - Endereço de memória onde o bloco deve ser armazenado;
    - Número de bytes a transferir.

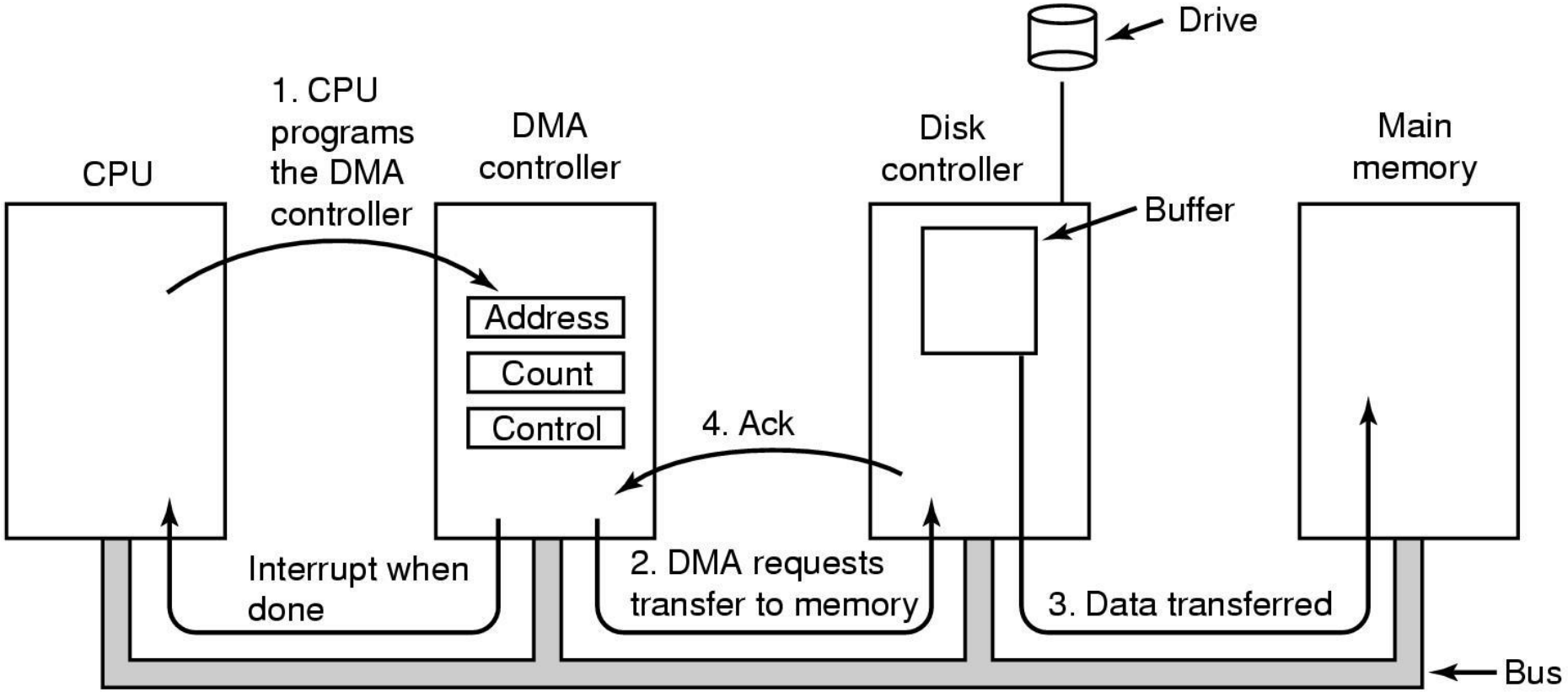
DMA
(Direct Memory Access)

- Para fazer DMA, a controladora precisa ter alguns elementos:
  - O buffer;
  - Um registrador que aponta o byte atual dentro do buffer;
  - Um registrador com o endereço de memória para transferir o byte atual;
  - Um contador de bytes a transferir.

DMA
(Direct Memory Access)

- Loop de transferência:
  - A controladora transfere o byte atual do buffer para a memória;
  - Incrementa os 2 registradores;
  - Subtrai 1 do contador;
  - Quando o contador chega a Zero, o processo para.
- Ao término da transferência, ocorre uma interrupção para avisar o SO que os dados estão no endereço dado!

DMA
(Direct Memory Access)



DMA
(Direct Memory Access)

Por que é preciso um buffer para a controladora?

- O barramento é único!
- O fluxo de bits do disco para a controladora não pode ser interrompido;
- Se a controladora vai armazenar um byte na memória e o barramento está ocupado por outra operação, ela tem que esperar!
- Isso pode levar a perda de bits do disco se a controladora não estiver pronta para recebê-los!

DMA
(Direct Memory Access)

- O buffer interno torna o projeto da controladora mais simples. <u>Consequências</u>:
  - Enquanto a transferência está sendo feita, o próximo setor pode passar pela cabeça de leitura!
  - Algumas controladoras não conseguem fazer entrada e saída simultaneamente:
    - Seriam necessários pelo menos 2 buffers;
    - Além disso, seria preciso um sistema de duplo controle (possivelmente paralelo)!
  - Neste caso, enquanto a controladora está transferindo dados para a memória, o próximo setor é perdido.

DMA
(Direct Memory Access)

- O acesso ao barramento pode ser de 2 tipos:

1. Uma palavra por vez:
   - A controladora de DMA pede uma palavra por vez;
   - Se nesse tempo a CPU quiser usar o barramento, ela espera;
   - Também chamado de ***cycle stealing***.

DMA
(Direct Memory Access)

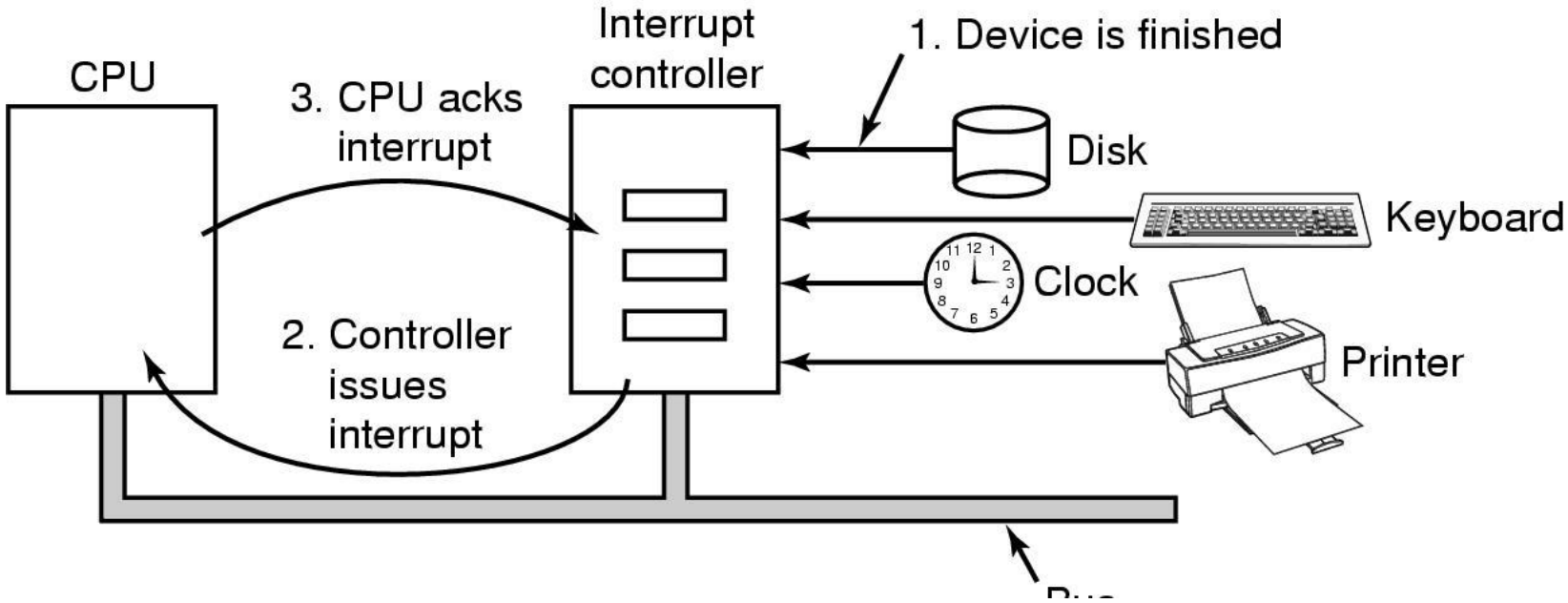
2. Por bloco:
   - O DMA diz ao dispositivo para adquirir o controle do barramento;
   - Várias transferências são feitas;
   - O barramento é liberado;
   - Mais eficiente, mas pode bloquear a CPU e outros dispositivos por tempo considerável.

DMA: Considerações

- Nem todos os computadores usam DMA.
- As CPUs atuais são mais rápidas que o DMA.
- Esta opção permite projetar hardware mais simples, o que é interessante em sistemas embutidos.

Interrupções de I/O

- Num computador típico, o sistema de interrupções funciona conforme a figura a seguir:



Interrupções de I/O

- Atualmente, usa-se linhas de interrupção no barramento, em vez de linhas específicas entre cada dispositivo e a controladora.
- A controladora de interrupção atende uma interrupção imediatamente, desde que não esteja ocupada.
- Se ela estiver tratando uma interrupção anterior, a atual é ignorada. Mas o dispositivo continua com o pedido ativo até ser atendido.

Interrupções de I/O

- Se duas interrupções chegam à controladora simultaneamente, aquela de maior prioridade será atendida primeiro. A prioridade é definida em geral por tipo de dispositivo.

- A controladora coloca um número nas linhas de endereço que indica qual dispositivo requer atenção e gera um sinal que interrompe a CPU.

Interrupções de I/O

- O sinal faz com que CPU pare o que está fazendo e passe a fazer outra coisa:
  - O número nas linhas de endereço é usado como um índice numa tabela chamada de **vetor de interrupção**;
  - Desta tabela se tira um endereço novo para o contador de instruções;
  - Esse endereço é o início da rotina de tratamento para aquele dispositivo;

Interrupções de I/O

- Uma das primeiras coisas que a rotina de tratamento faz é confirmar que recebeu a interrupção.
- Isso é feito colocando um valor em um dos registradores da controladora de interrupções.

Interrupções de I/O

- O hardware deve salvar certas informações antes de iniciar a rotina de tratamento.

- O que salvar depende da CPU. O mínimo seria o CI (Contador de Instruções), mas algum sistema pode exigir que todos os registradores sejam salvos.

- <u>Pergunta importante</u>: Onde salvar esses dados?

Interrupções de I/O

- Uma opção é colocar em registradores internos da controladora.
- Problema: A interrupção não pode ser confirmada até que esses dados todos sejam salvos (uma nova interrupção pode sobrescrever os dados da anterior).
- Isso causa períodos longos de tempo com interrupções desabilitadas, que podem causar perda de dados.

Interrupções de I/O

- Assim, a maioria das CPUs salva os dados em pilhas de execução.

- Uma interrupção obriga a CPU a perder pipelines, invalidar caches, memória associativa, etc.
  - Porque? É necessária a troca de contexto para a execução de uma rotina de tratamento.

Interrupções de I/O

- Outro problema: A instrução corrente não pode ser definida com precisão. Há circuitos de processamento paralelo, instruções sendo executadas adiantadas, etc.
  - Exemplo: Predição de Saltos.

- Salvar que contexto, neste caso?

Interrupções Precisas

- Uma interrupção que deixa a máquina num estado bem definido é chamada de **interrupção precisa**. Para tanto, é preciso atender 4 propriedades:
  - O CI é salvo em lugar conhecido;
  - Todas as instruções anteriores àquela apontada pelo CI foram executadas completamente;
  - Nenhuma instrução depois daquela apontada pelo CI foi executada;
  - O estado de execução da instrução apontada pelo CI é conhecido (Pode ou não ter sido executada).

Interrupções Imprecisas

- Uma interrupção que não atende esses requisitos é chamada de **interrupção imprecisa**:
  - São bastante desagradáveis para um projetista de SO;
  - Essas máquinas, em geral, guardam um grande volume de dados sobre o estado interno do sistema na pilha;
  - O SO deve varrer esses dados e determinar o que acontece;
  - Salvar e tratar esses dados em cada interrupção toma muito tempo;
  - Isso leva a um contrassenso interessante: CPUs muito rápidas às vezes não são apropriadas para aplicações de tempo real devido a sistema de interrupções lento.

Dúvidas?